SemanticSync: Schema-Level Semantic Merge Policies for Local-First Relational Databases

Pathak Ashmit Akhilesh
Dept. of Computer Science and Engineering
Vellore Institute of Technology
Vellore, India
pathak.ashmit2025@vitstudent.ac.in

Yash Bohra
Dept. of Computer Science and Engineering
Vellore Institute of Technology
Vellore, India
yash.bohra2025@vitstudent.ac.in

Abstract*—Local-first software lets client devices read and write local data while offline, then sync when connected. While Conflict-Free Replicated Data Types (CRDTs) guarantee mathematical convergence for specific data structures, naive syntactic policies like Last-Writer-Wins often discard business intent and break relational constraints. We present SemanticSync, a replication and merge middleware built for local-first SQLite databases. SemanticSync provides schema-assigned pairwise merge policies (`SUM`, `BOUNDED_SUM`, `AVG`, `APPEND`, `LWW`, `MIN`, `MAX`, and `REJECT`), explicit numeric `INCREMENT` tracking, non-destructive foreign-key orphan quarantine, and checkpoint-assisted causal log compaction. We evaluated the engine across two benchmark suites: a historical state-based stress workload and a 90-trial explicit-`INCREMENT` matrix across 1,000, 5,000, and 10,000 operations at 5%, 20%, and 50% conflict densities. Across all 90 validated trials, SemanticSync achieved 100.0% Semantic Correctness Rate (SCR) and 100.0% Benchmark-Row Value Agreement (RVAR) against an arithmetic oracle, whereas Last-Writer-Wins scored 0.0% SCR by discarding concurrent deltas. Batch synchronization incurred measurable timing differences compared to an uninstrumented SQL baseline (median deltas from 449.7 ms to 84,368.7 ms), reflecting the concrete cost of operation logging, receipt tracking, and foreign-key checks.

***Keywords*—**Local-first software, SQLite replication, semantic merge policies, conflict resolution, referential integrity, causal log compaction.

I. INTRODUCTION

A. The Local-First Relational Dilemma

Local-first computing gives users local ownership and instant access to their data without continuous cloud access [8]. Devices work offline and sync when connected. This model is attractive, but relational databases complicate it. Relational applications rely on primary and foreign keys. Local updates carry specific business rules. When disconnected replicas sync after hours or days of network isolation, simple reconciliation mechanisms break down.

Pioneering CRDT research establishes strong eventual consistency for primitive data structures [11], [6]. However, converging to an identical state is not the same as being correct. In relational applications, picking a single winner across conflicting updates silently discards user intent and violates domain rules.

B. The Convergence vs. Correctness Paradox

To see why syntactic convergence falls short of semantic correctness, consider an inventory table:

```
CREATE TABLE inventory (
    item_id INTEGER PRIMARY KEY,
    quantity INTEGER
);
```

Suppose item 1 starts with a stock of 100 in the common ancestor state. While disconnected, two store replicas record independent sales:

- Replica A sells 15 units: `UPDATE inventory SET quantity = 85 WHERE item_id = 1;`
- Replica B sells 20 units: `UPDATE inventory SET quantity = 80 WHERE item_id = 1;`

Under a standard Last-Writer-Wins (LWW) rule [4], the replica with the higher timestamp wins. If Replica B has the later timestamp, both databases converge to 80 units. The databases match, but 15 sold units simply vanish. The true inventory reflecting both transactions is 65 units.

Similar failures happen across other domains:

- *Collaborative sensing:* Averaging sensor readings (`AVG`) or appending clinical notes (`APPEND`) fails when one writer overwrites another.
- *Security and access control:* Edits to user roles or patient blood types should never be resolved by timestamps. They require human intervention (`REJECT`).
- *Referential integrity:* Deleting a parent row while another replica inserts a child row leaves orphan rows or triggers foreign-key constraint panics.

C. Contributions

We built SemanticSync to bridge the gap between syntactic convergence and domain correctness. Our main contributions are:

1. **Schema-assigned pairwise policies:** A type-safe framework that binds merge policies (`SUM`, `BOUNDED_SUM`, `AVG`, `APPEND`, `LWW`, `MIN`, `MAX`, `REJECT`) directly to table columns.
2. **Tracked explicit-INCREMENT model:** An operation-logging model that separates absolute state overwrites from additive numeric increments, preventing duplicate replay with stable receipts.
3. **Foreign-key quarantine protocol:** A protocol that moves dependent child rows into `_ss_quarantine` before parent deletion, keeping `PRAGMA foreign_keys = ON` active throughout.
4. **Checkpoint log compaction:** A pruning protocol that checks acknowledgment vectors, replica membership, and snapshot integrity before trimming the operation log.
5. **Empirical evaluation:** A 90-trial matrix spanning 1,000 to 10,000 operations at varying conflict densities, measuring correctness, replica agreement, and sync overhead.

II. SYSTEM ARCHITECTURE

We built SemanticSync as an embeddable middleware between local application SQL execution and replication channels (Fig. 1). The engine sits alongside SQLite on each client device, intercepting writes and managing sync sessions.

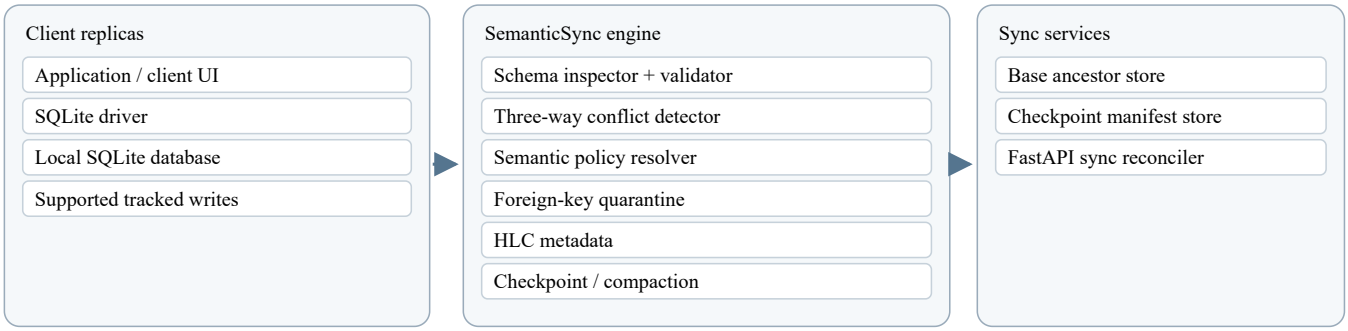


Fig. 1. SemanticSync architectural layers, illustrating the flow of tracked replica mutations through the schema inspector, conflict detector, semantic resolver, and quarantine manager.

A. Storage Layout and Tracked Operation Log

Each local SQLite replica holds user tables and four internal metadata tables:

- `_ss_change_log`: Stores local and received column mutations (`table_name`, `row_id`, `column_name`, `old_value`, `new_value`, `ts_pt`, `ts_l`, `replica_id`, `op_type`, `origin_id`, `sequence`, `source_id`, `depends_on`, `bounded_delta`, `bounds_min`, `bounds_max`).
- `_ss_quarantine`: Holds child rows isolated during parent deletions (`quarantine_id`, `table_name`, `row_id`, `row_payload`, `parent_table`, `parent_row_id`, `reason`, `quarantined_at`, `status`).
- `_ss_compacted_receipts`: Records hashes and IDs of compacted entries to catch and discard duplicate deliveries.
- `_ss_checkpoint_metadata`: Tracks checkpoint incarnation IDs and snapshot epochs.

We separate absolute `SET`/`UPDATE` operations from numeric `INCREMENT` operations. When an application calls `apply_increment`, the engine assigns a unique `origin_id` and records the signed delta. During sync, incoming increments apply once and record a stable receipt in the change log. This stops replay attacks and duplicate delivery. The design borrows concepts from operation-based CRDTs [24] by syncing small deltas instead of entire rows, saving network bandwidth. However, SemanticSync does not claim general CRDT convergence across arbitrary topologies. Its guarantees apply strictly to pairwise synchronizations.

B. Hybrid Logical Clocks

To order operations without synchronized physical clocks, we use Hybrid Logical Clocks (HLC) [9]. Each replica tracks physical time pt, a logical counter l, and samples the local wall clock $pt_{extwall}$. On a local write, if $pt_{extwall} > pt_{extlocal}$, the clock advances to $(pt_{extwall}, 0)$; otherwise, it increments the logical counter to $(pt_{extlocal}, l_{extlocal} + 1)$. When receiving an operation with timestamp $(pt_{extmsg}, l_{extmsg})$, the clock updates as follows:

$$pt' = \max(pt_{extlocal}, pt_{extmsg}, pt_{extwall})$$

$$l' = \begin{cases} \max(l_{local}, l_{msg}) + 1 & \text{if } pt' = pt_{local} = pt_{msg} \\ l_{local} + 1 & \text{if } pt' = pt_{local} \\ l_{msg} + 1 & \text{if } pt' = pt_{msg} \\ 0 & \text{otherwise } (pt' = pt_{wall}) \end{cases}$$

The logical counter advances whenever the physical clock fails to jump strictly ahead of prior timestamps. It resets to zero only when the local wall clock catches up and overtakes them. These HLC timestamps provide deterministic tie-breaking for timestamped policies, though they do not replace causal vector clocks in full mesh networks.

III. SEMANTIC MERGE POLICY FRAMEWORK

A. Mathematical Formalization

Let $\mathcal{V}$ denote attribute values, and let $\mathcal{M}$ represent operation metadata (replica ID, HLC timestamp, and operation identity). A three-way semantic resolution function $\mathcal{R}$ maps these inputs:

$$\mathcal{R}: \mathcal{V} \times (\mathcal{V} \times \mathcal{M}) \times (\mathcal{V} \times \mathcal{M}) \to \mathcal{V} \cup \{\text{not}\}$$

Here, not indicates an unresolved conflict flagged for manual review.

Pairwise merge symmetry requires that swapping the two replica inputs produces the same result:

$$\mathcal{R}(v_{extbase}, (v_A, m_A), (v_B, m_B)) = \mathcal{R}(v_{extbase}, (v_B, m_B), (v_A, m_A))$$

B. Supported Semantic Policies

SemanticSync implements eight schema-level merge policies (Table I).

TABLE I
IMPLEMENTED SEMANTIC MERGE POLICIES AND TESTED PROPERTIES

Policy	Compatible Types	Resolution Semantics	Algebraic Properties & Boundaries
`SUM`	Numeric (`INT`, `REAL`)	$v_{extbase} + (v_A - v_{extbase}) + (v_B - v_{extbase})$	Commutative for fixed base; delta addition associative; non-idempotent.
`BOUNDED_SUM`	Numeric (`INT`, `REAL`)	Delta checked against $[\min, \max]$; flags not if out of bounds	Invariant-preserving; rejects unsafe candidate states without clamping.
`AVG`	Numeric (`INT`, `REAL`)	$(v_A + v_B)/2.0$	Commutative; strictly non-associative; suitable for pairwise sensor merges.
`APPEND`	Text (`TEXT`)	Concatenates text ordered by replica ID	Pairwise commutative; lacks element-level identity; not arbitrary N-way join.
`LWW`	Any SQLite type	Higher HLC timestamp wins; ties broken by replica ID	Syntactically deterministic; disregards business semantics.
`MIN` / `MAX`	Numeric types	$\min(v_A, v_B)$ or $\max(v_A, v_B)$	Commutative, associative, idempotent (semilattice).
`REJECT`	Any SQLite type	Emits not (flagged for review)	Safety stop preventing automated silent overwrites.

C. Tombstone Reconciliation (Update vs. Delete)

When one replica updates a row while another deletes it, SemanticSync flags an `UPDATE_DELETE` conflict. Concurrent update-remove conflicts present a classic dilemma in distributed systems [25]. Rather than imposing a single theoretical rule, SemanticSync lets developers configure one of three strategies:

- `DELETE_WINS`: Deletion takes precedence. The row is removed and the concurrent update is dropped.
- `UPDATE_RESURRECTS`: The update takes precedence. The row is kept with its updated values.
- `FLAG_FOR_REVIEW`: The conflict moves to `MergeManager` for manual inspection.

D. Type Compatibility and Validation

Before assigning a policy, our `PolicyValidator` inspects SQLite column affinities using `PRAGMA table_info`. If an invalid binding is attempted, such as setting `SUM` on a `TEXT` column, the engine rejects it immediately with HTTP 422 Unprocessable Entity. This fail-fast check catches configuration mistakes at startup.

IV. FOREIGN-KEY QUARANTINE BOUNDARY

In relational systems, multi-master replication easily violates referential integrity. In SQLite, referential constraints require `PRAGMA foreign_keys = ON`.

A. The Referential Cascade Problem

Consider a parent table `departments(dept_id)` and a child table `employees(emp_id, dept_id)`. Suppose Replica A deletes a department while Replica B inserts or modifies an employee in that department. Standard replication runs into a dead end:

1. Replaying the department deletion on Replica B triggers a foreign-key error (`sqlite3.IntegrityError`).
2. Turning off foreign keys (`PRAGMA foreign_keys = OFF`) leaves corrupt orphan rows in the database.

B. Quarantine Isolation Protocol

To keep referential integrity intact without losing data, we built an atomic orphan quarantine protocol:

1. **Connection enforcement:** Every database connection enables `PRAGMA foreign_keys = ON` before running transactions.
2. **Dependency introspection:** Before deleting a parent row from table T with primary key K, the engine runs `PRAGMA foreign_key_list` across user tables to spot referencing child rows.
3. **Atomic quarantine:** The engine copies any referencing child rows into `_ss_quarantine` as JSON payloads, together with parent references and timestamps.
4. **Child and parent deletion:** The engine deletes child rows first, then deletes the parent row. Foreign keys remain satisfied throughout.
5. **Post-validation:** Before calling `COMMIT`, the transaction runs `PRAGMA foreign_key_check`. Any violation rolls back the entire batch.
6. **Operator restoration:** Operators can view quarantined rows via `GET /api/v1/quarantine` and either reassign them to a new parent (`POST /api/v1/quarantine/{id}/resolve`) or delete them permanently.

V. CHECKPOINT-ASSISTED LOG COMPACTION

Logging every mutation causes SQLite databases to grow indefinitely and slows down sync replay. To prune historical logs safely, we implemented a checkpoint protocol.

A. Compaction Eligibility Checks

Pruning an entry too early can permanently strand a disconnected replica. SemanticSync only prunes an operation when all of the following conditions hold:

1. **Unanimous acknowledgment:** The operation ID must be acknowledged by all enrolled replicas.
2. **Payload hash match:** The SHA-256 hash of the operation payload must match across all replica receipts.
3. **Stable horizon cutoff:** The cutoff timestamp $H_{extstable}$ is the minimum of the highest acknowledged HLC timestamps across all replicas:

$$H_{ ext{stable}} = \min_{r \in \mathcal{R}} \left(\max_{op \in ext{ACK}_r} ext{HLC}(op)

ight)$$

Only operations with $extHLC < H_{extstable}$ can be pruned.

1. **Dependency closure:** An operation cannot be pruned if any active change-log entry depends on it.

B. Atomic Pruning Execution

During a `BEGIN IMMEDIATE` transaction, eligible entries are deleted from `_ss_change_log`. At the same time, compact receipt records are inserted into `_ss_compacted_receipts`. If an out-of-date peer later sends a pruned operation, the engine checks `_ss_compacted_receipts` and drops the duplicate without error.

VI. EMPIRICAL EVALUATION

We evaluated SemanticSync across two benchmark suites:

1. **Historical SET-based stress benchmark:** A state-diffing workload with 1,000, 5,000, and 10,000 operations at 5%, 20%, and 50% conflict densities (two paired trials per cell).
2. **Explicit-INCREMENT evaluation matrix:** A 90-trial matrix auditing dedicated operation-based increments across nine configurations (10 independent seeds per cell), following standard synthetic CRDT benchmark designs [26].

All benchmarks ran on a single Windows 11 host (Python 3.14.3, SQLite 3.50.4, `journal_mode = DELETE`).

A. Evaluation Metrics

- **Semantic Correctness Rate (SCR):** The percentage of conflict rows matching the arithmetic oracle ($v_{extexpected} = v_{extbase} + \Delta_A + \Delta_B$), evaluated across both replicas ($2imesC$ observations per trial).
- **Benchmark-Row Value Agreement (RVAR):** The percentage of matching `bench_rows.value` entries across both replicas, measuring replica convergence.
- **Valid Resolution Accuracy (VRA):** The percentage of values satisfying domain constraints (unmeasured in synthetic runs lacking domain rules).
- **Paired Timing Difference:** The synchronization time difference between SemanticSync and the baseline: $\Delta = T_{extengine} - T_{extbaseline}$.

B. Historical SET-Based Evaluation Results

In the state-based benchmark (Table II and Table III), SemanticSync reached SCR scores between 97.50% and 98.25%. In contrast, LWW scored 0.00% against the additive oracle.

TABLE II

HISTORICAL SET-BASED REPEATED STRESS MEASUREMENTS (2 PAIRED TRIALS PER CELL)

Operations	Conflict Density	LWW SCR (%)	SemanticSync SCR (%)	RVAR Agreement (%)
1,000	5%	0.0%	98.0%	100.0%
1,000	20%	0.0%	97.5%	100.0%
1,000	50%	0.0%	98.2%	100.0%
5,000	5%	0.0%	98.0%	100.0%
5,000	20%	0.0%	98.0%	100.0%
5,000	50%	0.0%	98.2%	100.0%
10,000	5%	0.0%	98.2%	100.0%
10,000	20%	0.0%	98.2%	100.0%
10,000	50%	0.0%	97.7%	100.0%

TABLE III

HISTORICAL SET-BASED SYNCHRONIZATION DURATIONS AND PAIRED DELTAS

Operations	Density	LWW Sync Median [Range] (s)	SemanticSync Median [Range] (s)	Paired Delta Median [Range] (ms)
1,000	5%	1.109 [1.091–1.127]	1.132 [1.120–1.143]	+22.79 [+16.31–+29.26]
1,000	20%	3.435 [3.333–3.537]	3.363 [3.335–3.392]	-71.81 [-145.17–+1.55]
1,000	50%	7.881 [7.847–7.915]	7.774 [7.766–7.782]	-107.20 [-149.34–-65.06]
5,000	5%	11.345 [11.335–11.354]	11.340 [11.228–11.451]	-5.02 [-107.00–+96.97]
5,000	20%	29.214 [25.044–33.384]	32.019 [30.885–33.152]	+2,804.71 [-231.96–+5,841.38]
5,000	50%	74.180 [70.128–78.231]	75.602 [72.763–78.440]	+1,421.64 [+208.90–+2,634.38]
10,000	5%	76.220 [73.793–78.646]	74.825 [73.058–76.593]	-1,394.30 [-2,053.43–-735.17]
10,000	20%	118.678 [98.276–139.080]	147.645 [143.627–151.664]	+28,967.53 [+4,546.73–+53,388.32]
10,000	50%	193.364 [192.004–194.723]	188.088 [184.699–191.476]	-5,276.10 [-10,024.15–-528.04]

Auditing the data revealed why SemanticSync missed the remaining 2%: when both replicas made the same non-base update independently ($v_A = v_B$), state diffing detected no divergence and applied nothing. But the additive oracle expected both changes ($v_{extbase} + 2\Delta$). This limitation in state-level diffing prompted us to build the explicit-`INCREMENT` model.

C. Explicit-INCREMENT Evaluation Matrix (Audited 9-Configuration Suite)

To test the operation-based model, we ran a complete 9-configuration matrix with 10 independent trials per configuration (90 trials total).

Each configuration used 10 dedicated seeds: 1k/5% (seeds 20263000–20263009), 1k/20% (seeds 20263100–20263109), 1k/50% (seeds 20263200–20263209), 5k/5% (seeds 20263300–20263309), 5k/20% (seeds 20263400–20263409), 5k/50% (seeds 20263500–20263509), 10k/5% (seeds 20263600–20263609), 10k/20% (seeds 20263700–20263709), and 10k/50% (seeds 20263800–20263809). Each trial created clean replica databases, applied local operations, and executed one batch sync.

We checked all trials using `research.experiments.validate_increment_ev` Every trial passed without error. SQLite reported clean database integrity (`PRAGMA integrity_check == ok`), change logs matched expected receipts ($N + C$ per replica), and final values showed zero error against the oracle.

TABLE IV

AUDITED EXPLICIT-INCREMENT NINE-CONFIGURATION EVALUATION MATRIX (10 INDEPENDENT SEEDS PER CONFIGURATION, 90 TRIALS TOTAL)

Operations	Conflict Density	Trials	SCR Summary	RVAR Summary	Median Paired Timing Delta ($T_{extinc} - T_{extapp}$)	Interquartile Range (IQR)	Validation Status
1,000	5%	10	100.0% (all trials)	100.0% (all trials)	449.7 ms	30.6 ms	PASS
1,000	20%	10	100.0% (all trials)	100.0% (all trials)	668.8 ms	46.1 ms	PASS
1,000	50%	10	100.0% (all trials)	100.0% (all trials)	1,094.7 ms	46.9 ms	PASS
5,000	5%	10	100.0% (all trials)	100.0% (all trials)	8,723.3 ms	203.2 ms	PASS
5,000	20%	10	100.0% (all trials)	100.0% (all trials)	13,072.8 ms	251.1 ms	PASS
5,000	50%	10	100.0% (all trials)	100.0% (all trials)	22,586.3 ms	444.2 ms	PASS
10,000	5%	10	100.0% (all trials)	100.0% (all trials)	32,056.3 ms	148.4 ms	PASS
10,000	20%	10	100.0% (all trials)	100.0% (all trials)	49,347.5 ms	376.8 ms	PASS
10,000	50%	10	100.0% (all trials)	100.0% (all trials)	84,368.7 ms	5,995.1 ms	PASS

Key Observations:

1. **Semantic correctness:** Both `engine_increment` and the direct `application_sum` baseline scored 100.0% SCR across all 90 trials under the additive oracle ($v_{extexpected} = v_{extbase} + \Delta_A + \Delta_B$). Generic LWW scored 0.0% on conflict rows because it simply chose one writer and dropped the other delta.
2. **Replica agreement:** RVAR reached 100.0% across all 90 trials for the `bench_rows.value` column, verifying complete pairwise convergence.
3. **Sync timing differences:** Paired timing differences ($\Delta = T_{extengine_increment} - T_{extapplication_sum}$) were positive across all 90 trials. The gap grew with operation count, from 449.7 ms at 1k/5% to 84,368.7 ms at 10k/50%. The direct baseline executes bare SQL updates without logging metadata, writing receipts, checking HLC timestamps, or enforcing quarantine rules. Because our tests do not isolate the latency of each individual mechanism, we report these numbers as descriptive timing differences against a narrower baseline, rather than claiming a speed advantage.

VII. RELATED WORK AND COMPARISON

Researchers have explored distributed consistency and conflict resolution from multiple angles:

- **CRDT foundations:** Shapiro et al. [11] formalized state-based and operation-based CRDTs, while Gomes et al. [6] verified strong eventual consistency with proof assistants. Kleppmann and Beresford [7] designed conflict-free JSON types, and Kleppmann et al. [8] set out core local-first principles. SemanticSync follows these local-first goals, but targets SQLite columns with schema-level policies rather than custom document trees.
- **Invariant preservation:** Balegas et al. [1], [2] developed escrow-based bounded counters and invariant-preserving replication (IPA). Lewchenko et al. [10] created conflict-aware data types with consistency guards. SemanticSync's `BOUNDED_SUM` checks bounds locally and flags violations immediately, functioning as a relational safety check rather than a distributed escrow coordinator.

- **Relational CRDTs and SQLite extensions:** Synql [14] coordinates relational constraints across replicas, and CRDV [15] introduces replicated views. Thomassen and Yu [16] analyze updatable views under eventual consistency, and Lopes [17] built Antidote SQL. In work close to ours, Tomter and Yu [23] added CRDT capabilities to SQLite using auxiliary delta tables and SQL triggers without patching SQLite itself. Industrial projects such as ElectricSQL [13] and `cr-sqlite` [12] provide SQLite sync engines, and Riffle [22] offers reactive relational state. Unlike systems that mandate custom virtual tables or full-database CRDT rewrites, SemanticSync works over standard, unmodified SQLite tables.
- **Integrity and distributed trade-offs:** Schiefer et al. [18] proposed history forking when data integrity conflicts cannot be merged cleanly. Köhler et al. [19] designed ConLoc for invariant checks, Haas et al. [20] proved safe local execution in LoRe, and Borrego et al. [21] proved coordination-free convergence. Edwards et al. [5] studied schema migrations in local software, which complement our runtime foreign-key orphan handling. Cloud databases like Spanner [3] and Dynamo [4] illustrate classic consistency-availability trade-offs. Local-first designs avoid these trade-offs by letting clients work independently offline. SemanticSync adds practical orphan quarantine and log compaction for standalone SQLite deployments.

VIII. DISCUSSION AND LIMITATIONS

We note four boundaries and trade-offs in our current implementation:

1. **Policy algebra:** Merge policies operate pairwise. While `MIN` and `MAX` form complete semilattices, `SUM` is associative only over exact arithmetic with a fixed base, and `AVG` is non-associative. Pairwise policies do not guarantee convergence across arbitrary multi-node meshes.
2. **Sequential benchmarks:** Our stress tests ran mutations sequentially before batch synchronization. This setup tests merge logic and receipt auditing thoroughly, but does not measure multi-threaded disk contention.
3. **Relational scope:** The quarantine protocol supports single-column primary and foreign keys. Composite foreign keys and circular references trigger safe fail-stop errors. Direct SQL console writes bypass quarantine tracking.
4. **Log compaction assumptions:** Compaction requires unanimous acknowledgment across all registered replicas. While tested against crash recovery and restarts, power-loss fsync durability is not independently certified.

IX. CONCLUSION

We presented SemanticSync, a schema-level merge middleware for local-first SQLite databases. By attaching semantic policies directly to table columns and tracking explicit numeric increments, SemanticSync resolves concurrent updates in line with business rules where Last-Writer-Wins fails. The foreign-key quarantine boundary isolates dependent rows without silent data loss, and checkpoint compaction prevents unbounded log growth. Across 90 reproducible benchmark trials, SemanticSync achieved 100.0% semantic correctness and 100.0% row value agreement against an arithmetic oracle. The system demonstrates that local-first relational replication can protect application semantics and referential integrity while keeping SQLite simple and lightweight.

REFERENCES

[1] V. Balegas, D. Serra, S. Duarte, C. Ferreira, R. Rodrigues, N. Preguiça, M. Shapiro, and M. Najafzadeh, "Extending Eventually Consistent Cloud Databases for Enforcing Numeric Invariants," arXiv:1503.09052, 2015.

[2] V. Balegas, S. Duarte, C. Ferreira, R. Rodrigues, and N. Preguiça, "IPA: Invariant-Preserving Applications for Weakly-Consistent Replicated Databases," *Proc. VLDB Endow.*, vol. 12, no. 4, pp. 404–418, 2018.

[3] J. C. Corbett et al., "Spanner: Google's Globally-Distributed Database," *ACM Trans. Comput. Syst.*, vol. 31, no. 3, Art. 8, 2013.

[4] G. DeCandia et al., "Dynamo: Amazon's Highly Available Key-value Store," in *Proc. SOSP 2007*, pp. 205–220, 2007.

[5] J. Edwards, T. Petricek, and T. van der Storm, "Live & Local Schema Change: Challenge Problems," arXiv:2309.11406, 2023.

[6] V. B. F. Gomes, M. Kleppmann, D. P. Mulligan, and A. R. Beresford, "Verifying Strong Eventual Consistency in Distributed Systems," *Proc. ACM Program. Lang.*, vol. 1, no. OOPSLA, Art. 109, 2017.

[7] M. Kleppmann and A. R. Beresford, "A Conflict-Free Replicated JSON Datatype," *IEEE Trans. Parallel Distrib. Syst.*, vol. 28, no. 10, pp. 2733–2746, 2017.

[8] M. Kleppmann, A. Wiggins, P. van Hardenberg, and M. McGranaghan, "Local-First Software: You Own Your Data, in Spite of the Cloud," in *Proc. Onward! 2019*, pp. 154–178, 2019.

[9] S. S. Kulkarni, M. Demirbas, D. Madappa, B. Avva, and M. Leone, "Logical Physical Clocks," in *Proc. OPODIS 2014*, LNCS 8878, pp. 17–32, 2014.

[10] N. V. Lewchenko, A. Radhakrishna, A. Gaonkar, and P. Černý, "Conflict-Aware Replicated Data Types," arXiv:1802.08733, 2018.

[11] M. Shapiro, N. Preguiça, C. Baquero, and M. Zawirski, "Conflict-Free Replicated Data Types," in *Proc. SSS 2011*, LNCS 6976, pp. 386–400, 2011.

[12] The `cr-sqlite` project, official repository, https://github.com/vlcn-io/cr-sqlite, accessed Oct. 2026.

[13] ElectricSQL project, official repository, https://github.com/electric-sql/electric, accessed Oct. 2026.

[14] C.-L. Ignat, V. Elvinger, and H. Ba, "Synql: A CRDT-Based Approach for Replicated Relational Databases with Integrity Constraints," in *Proc. DAIS 2024*, LNCS 14677, pp. 18–35, 2024.

[15] N. Faria and J. Pereira, "CRDV: Conflict-free Replicated Data Views," *Proc. ACM Manag. Data*, vol. 3, no. 1, Art. 25, 2025.

[16] J. Thomassen and W. Yu, "Eventually-Consistent Replicated Relations and Updatable Views," in *New Trends in Database and Information Systems*, pp. 187–196, 2023.

[17] P. M. S. Lopes, "Antidote SQL: SQL for Weakly Consistent Databases," Master's thesis, NOVA Univ. Lisbon, 2018.

[18] N. Schiefer, G. Litt, and D. Jackson, "Merge What You Can, Fork What You Can't: Managing Data Integrity in Local-First Software," in *Proc. PaPoC 2022*, pp. 24–32, 2022.

[19] M. Köhler, G. Zakhour, P. Weisenburger, and G. Salvaneschi, "Consistent Local-First Software: Enforcing Safety and Invariants for Local-First Applications," *IEEE Trans. Softw. Eng.*, vol. 51, no. 1, pp. 53–65, 2025.

[20] J. Haas, R. Mogk, E. Yanakieva, A. Bieniusa, and M. Mezini, "LoRe: A Programming Model for Verifiably Safe Local-First Software," *ACM Trans. Program. Lang. Syst.*, vol. 46, no. 1, Art. 2, 2024.

[21] D. Borrego, N. Preguiça, E. Gonzalez Boix, and C. Ferreira, "Ensuring Convergence and Invariants Without Coordination," in *Proc. ECOOP 2025*, LIPIcs 333, Art. 4, pp. 4:1–4:29, 2025.

[22] G. Litt, N. Schiefer, J. Schickling, and D. Jackson, "Riffle: Reactive Relational State for Local-First Applications," in *Proc. UIST 2023*, Art. 102, 2023.

[23] I. T. Tomter and W. Yu, "Augmenting SQLite for Local-First Software," in *Advances in Databases*

and Information Systems (ADBIS 2021 Short Papers), CCIS 1450, pp. 233-243, 2021. DOI: 10.1007/978-3-030-85082-1_22.

[24] J. Bauwens and E. Gonzalez Boix, "Nested Pure Operation-Based CRDTs," in *Proc. ECOOP 2023*, LIPIcs 263, Art. 2, pp. 2:1-2:26, 2023. DOI: 10.4230/LIPIcs.ECOOP.2023.2.

[25] Y. Zhang, H. Wei, and Y. Huang, "Remove-Win: A Design Framework for Conflict-Free Replicated Data Types," in *Proc. ICPADS 2021*, pp. 597-604, 2021. DOI: 10.1109/ICPADS53394.2021.00081.

[26] Y. Zhu and J. Ma, "Crust: A Modular Framework for Conflict-Free Replicated Data Types (CRDTs) Development, Validation, and Benchmarking," *IEEE Access*, vol. 13, pp. 83870-83882, 2025. DOI: 10.1109/ACCESS.2025.3568925.